DOCUMENTO TÉCNICO · CORE FEATURES · OUTUBRO 2026

VAGA — Arquitetura das Funcionalidades Centrais

Como estão construídas, por dentro, as peças mais importantes da plataforma.

Fundação & princípios

Laravel 12 (PHP 8.4) · MySQL 8.4 · Redis · Meilisearch, tudo em **Docker**. Site **SSR (Blade)** + **API REST** versionada (`/api/v1`) a partir do mesmo domínio.

- **Lógica de negócio em serviços/ações** (SOLID), controladores finos separados por canal (`Web` / `Api\V1`).
- **Enums** tipados para estados/papéis; **Policies** para autorização; **Form Requests** para validação.
- Qualidade imposta a cada alteração: **Pint** (estilo), **Larastan nível 6** (análise estática) e **170 testes** (Pest).
- Padrão recorrente de **abstração por contrato + driver** (pesquisa, push, pagamentos): troca-se a implementação sem mexer na lógica, e testa-se sem serviços externos.

01 Pesquisa PRODUÇÃO

Rápida, tolerante a erros e multilingue — via **Laravel Scout + Meilisearch**.

- **Índice por evento** (`toSearchableArray`): título, resumo e descrição **achatados nas 6 línguas**, mais categorias (todos os idiomas), tags, promotor e locais. Só eventos publicados são indexados (`shouldBeSearchable`).
- **Sincronização automática** ao criar/editar/despublicar; carga inicial via `scout:import` .
- O motor dá **typo-tolerance**, pesquisa por prefixo (as-you-type) e ranking por relevância; suporta sinónimos.

> **Separação de responsabilidades:** o Meilisearch decide a **relevância** (devolve ids ordenados); a MySQL continua a ser a **fonte de verdade** e aplica as regras (publicado + ocorrência futura). Em testes usa-se o driver `collection` (sem Meilisearch). Verificado: `"muzica rok"` → "Concerto de Música Rock".

02 Recomendações `PRODUÇÃO (Fase 1)`

Personalização baseada em conteúdo, com explicação ("porquê").

O `RecommendationService` pontua os próximos eventos publicados pelos sinais do utilizador e devolve, em cada um, a **razão** mais forte.

SINAL	PESO	RAZÃO APRESENTADA
Promotor seguido	6	"Porque segues X"
Organização seguida	5	"De uma organização que segues"
Categoria favorita	4	"Porque gostas de Jazz"
Tag favorita	3	"Porque te interessa #ar-livre"
Histórico (favoritos/agenda)	2	"Parecido com o que guardaste"
Em destaque	1	"Em destaque"

> **Cold start:** sem sinais suficientes, recorre a eventos em destaque/mais próximos. Exclui o que já está na agenda. Alimenta o ecrã "Para ti", a API e o e-mail semanal. **Fase 2 (futuro):** filtragem colaborativa ("quem foi a isto também foi a…").

03 Internacionalização (6 idiomas) `PRODUÇÃO`

PT (base), EN, FR, ES, DE, IT.

- **Conteúdo traduzível** (título/descrições do evento, nome da categoria) em colunas JSON via `spatie/laravel-translatable`, com **fallback** para o idioma base.
- **Idioma por pedido** resolvido por middleware: `?lang=` → sessão → `Accept-Language` → default; funciona em web e API.

04 Bilhética & inventário `PRODUÇÃO (sem Stripe)`

Bilhetes gratuitos e pagos, com controlo de lotação e entrada.

- **Reserva atómica à prova de overselling**: ao inscrever/comprar, os tipos de bilhete são bloqueados na transação (`lockForUpdate`) e a capacidade é verificada — nunca se vende a mais, mesmo com acessos concorrentes.
- **Bilhete individual com código único** e **QR** (SVG); **check-in** valida e marca como usado (uso único garantido, também sob concorrência).
- **Estados** da encomenda: `free · pending · paid · expired · refunded`.

05 Pagamentos & comissão LÓGICA PRONTA STRIPE POR LIGAR

Modelo de marketplace: o promotor recebe, a VAGA retém comissão.

- **Comissão em cascata** (`CommissionResolver`): taxa do **evento** → senão do **promotor** → senão o **default** da plataforma.
- **Hold com validade**: a lotação fica reservada 15 min enquanto o pagamento está pendente; um **job agendado** (`tickets:expire-holds`) liberta reservas não pagas.
- Confirmação **idempotente** (preparada para o webhook): marcar como pago só emite uma vez; reembolso devolve lotação e reverte comissão.

> **Gateway abstraído** (contrato `PaymentGateway`): hoje corre um `FakePaymentGateway` (demonstração, testável sem credenciais). Ligar o Stripe = implementar `StripePaymentGateway` (destination charges + application fee) + onboarding Connect + webhook, trocando `PAYMENT_GATEWAY=stripe` . O fluxo não muda.

06 Suporte móvel & notificações BACKEND PRONTO

- **Registo de dispositivos** (tokens iOS/Android), idempotente, removido no logout.
- **Envio de push abstraído** (contrato `PushSender` + drivers `log` / `FCM`): gatilho real — ao publicar um evento, os seguidores do promotor/organização são notificados.
- **Feed agregado** (`/me/feed`): home da app numa só chamada (Para ti / Em destaque / A não perder).

07 Autenticação & autorização PRODUÇÃO

- **Web** (sessão) e **API** (tokens Sanctum). Toda a API de dados exige token (exceto registo/login).
- **Gate de registo**: lista/calendário públicos; detalhe de evento exige conta — em web e API.
- **Papéis** (user / promoter / organization / admin) com **Policies**; **impersonação** segura de promotor/organização pelo admin.

08 Retenção por e-mail PRODUÇÃO

- **Digest semanal personalizado** ("o que não podes perder"), que reaproveita o motor de recomendações — agendado.
- **Incentivo a promotores** sem eventos agendados; preferências e **opt-out** por utilizador; envios auditados.

09 Business Intelligence FUNDAÇÃO + ROADMAP

Entender os dados e partilhar *insights* com os promotores.

- **Já em recolha**: sinais de utilizador (pesquisas, visualizações de evento) num registo de atividade, mais os dados de primeira classe (favoritos, agenda, seguidores, bilhetes/check-ins).
- **Planeado**: inquéritos pós-evento ao promotor (nº de pessoas real vs. estimado), métricas de público e um painel de partilha com promotores.

> **Nota de estado:** a **recolha** de dados está ativa; os painéis e inquéritos de BI são o próximo módulo a desenhar — ainda não implementados.